

# Lesson 16 · Documentation from Sprint Changes

---

**PDF:** [this lesson](#) · **Install (Linux · macOS · Windows):** [guide](#) · [PDF](#)

**Part of [local-ai-lab](#)** - a hands-on course for building local AI.

**Course home:** <https://nikolareljin.github.io/local-ai-lab/> **Source:** <https://github.com/nikolareljin/local-ai-lab>

**Lessons:** [1 · RAG](#) → [2 · MCP](#) → [3 · Hybrid retrieval](#) → [4 · RAG safety](#) → [5 · RAG evaluation](#) → [6 · Repo assistant](#) → [7 · LangChain](#) → [8 · LangGraph](#) → [9 · Ollama tools](#) → [10 · Jev / System One](#) → [11 · Semantic Kernel](#) → [12 · Bedrock Agents](#) → [13 · Google ADK](#) → [14 · AI-assisted testing](#) → [15 · AI code review](#) → **16 · Docs from changes (you are here)**

**Status: planned.** Outline below; full step-by-step coming later. Star the repo to follow along.

---

## The idea

---

Point a model at a sprint's commits and ask for release notes and you get a tidied-up list of commit subjects. That is not documentation. **A changelog written from commit subjects is a list of commit subjects**, and the reader wanted to know what changed **for them**.

Getting further means giving the pipeline more than the log: the diff, the pull request discussion, and the issue it closed - which together carry the **why** that a commit subject drops. And it means letting the pipeline **refuse**, because a sprint full of refactors and dependency bumps has no user-visible changes, and saying so is the correct output rather than a failure.

## What you'll learn

---

- **Source across commits, pull requests and issues** - the reason for a change is almost never in the commit message
- **Group by audience, not by file** - "what changed for an API consumer" is a useful heading; "changes in `src/utils/`" is not
- **The no-user-visible-change filter** - the hardest and most valuable step, and the one every tool in this space skips
- **Cite back to pull request numbers** - the course's citation contract, one last time: every claim in the notes traceable to the change that produced it
- **Write into the format the repo already has** - a generator that invents its own changelog shape will be quietly reverted by the first human who edits it

## Builds on

---

| Concept                                         | From                        |
|-------------------------------------------------|-----------------------------|
| grounded answers with citations                 | <a href="#">Lesson 1</a>    |
| repo-aware retrieval                            | <a href="#">Lesson 6</a>    |
| "is the output any good?" as a tracked number   | <a href="#">Lesson 5</a>    |
| <b>refusing to describe what did not change</b> | <b>Lesson 16 (this one)</b> |

**The worked example is this repository.** `CHANGELOG.md` and `.github/workflows/release.yml` already turn a changelog section into a tagged release - so the last thing the course does is generate the input to a pipeline you have been reading since Lesson 1.

## Prerequisites

---

[Lesson 1](#), and a repository with a few sprints of history. Language-agnostic: the input is your version control, not your source.

## The end of the course

---

That is Lesson 16, and the end of the curriculum. Go back to the [syllabus](#) for the whole arc - or pick the one thing here you would actually use, and point it at your own repository. The last exercise in every lesson has been the same one.

---

**Course:** [nikolareljin.github.io/local-ai-lab](https://nikolareljin.github.io/local-ai-lab) · **Author:** [Nik Reljin](#)